

Hadoop与Spark

Spark简介



Spark简介

◆ Spark简介

- ◆ Apache Spark是一个开源的大数据处理框架，它与Hadoop并驾齐驱，是当前主流的大数据处理框架之一
- ◆ Spark于2009年由UC Berkeley的AMP实验室开发，并且在2010年开源，成为一个Apache项目
- ◆ Spark是一个速度快、易用、通用的集群计算系统，能够与Hadoop生态系统和数据源良好地兼容
- ◆ 目前，很多机构和组织已经在大规模集群上(上千节点)运行Spark，对他们的数据进行分析
- ◆ 根据Spark FAQ(Frequently Asked Questions)，到目前为止，实际部署的最大的集群，达到8000个节点
- ◆ Spark平台的大部分用Scala编写，有一小部分用Java编写

Spark简介

- ◆ **Spark简介**
 - ◆ 用户可以使用Shell程序，交互式地对数据进行查询
 - ◆ 除了简单的Map和Reduce操作，Spark本身提供了超过80个数据处理的原语操作(Primitive Operator)，方便用户编写数据处理程序
 - ◆ Spark提供了Java、Scala、以及Python语言的应用程序编程接口(API)
 - ◆ 用户可以使用这些操作，完成SQL查询、流数据处理、机器学习、以及图数据处理，还可以在一个工作流(Workflow)中，把这些功能整合起来

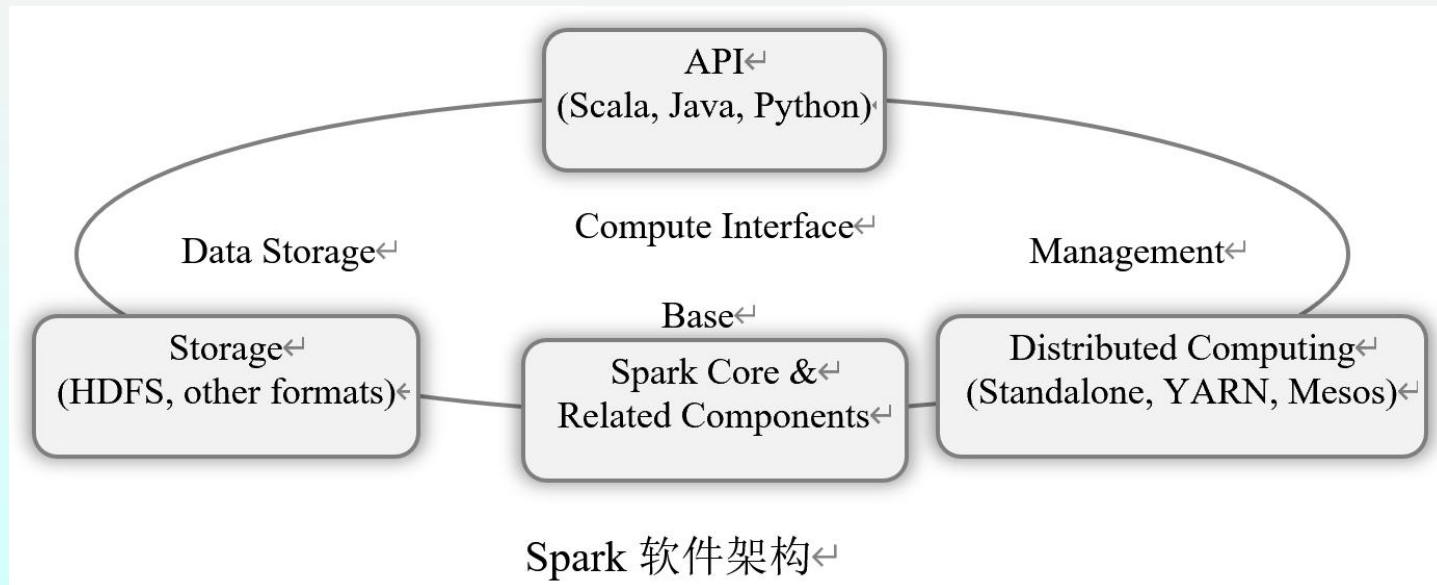
pause



Spark简介

- ◆ Spark软件架构

- ◆ 包含4个主要部分，分别是Spark的核心组件和相关组件、数据存储(Data Storage)、应用程序编程接口(API)、和资源管理框架(Management Framework)
- ◆ 其中，Spark的核心组件和相关组件，包括Spark Core，以及Spark Streaming、Spark SQL、GraphX、MLLib等组件



Spark简介

◆ Spark简介

- ◆ 2014年，Spark赢得了Daytona Gray Sort比赛(<http://sortbenchmark.org/>)
- ◆ Databricks公司(对Spark进行商业化的公司)的研究人员，在206台EC2虚拟机上，用23分钟，完成了100TB数据的排序
- ◆ 之前，Hadoop(MapReduce)创造的100TB数据的排序记录，是使用了2100台机器，并且耗费了72分钟完成的；排序是在磁盘数据上进行的(HDFS)，没有使用内存Cache
- ◆ 这个结果显示，Spark能够使用更少的硬件资源(少10倍)，对数据进行更快(快3倍)的排序；赢得这个比赛，是Spark发展的重要的里程碑

和 Spark100TB 数据排序结果的比较(数据来源 Databricks)

	Hadoop MR Record	Spark Record	Spark 1 PB
Data Size	102.5 TB	100 TB	1000 TB
Elapsed Time	72 mins	23 mins	234 mins
#Nodes	2100	206	190
#Cores	50400 physical nodes	6592 virtualized nodes	6080 virtualized nodes
Cluster disk throughput	3150 GB/s (est.)	618 GB/s	570 GB/s
Sort Benchmark Daytona Rules	Yes	Yes	No
Network	dedicated data center, 10Gbps	virtualized (EC2) 10Gbps network	virtualized (EC2) 10Gbps network
Sort rate	1.42 TB/min	4.27 TB/min	4.27 TB/min
Sort rate/node	0.67 GB/min	20.7 GB/min	22.5 GB/min

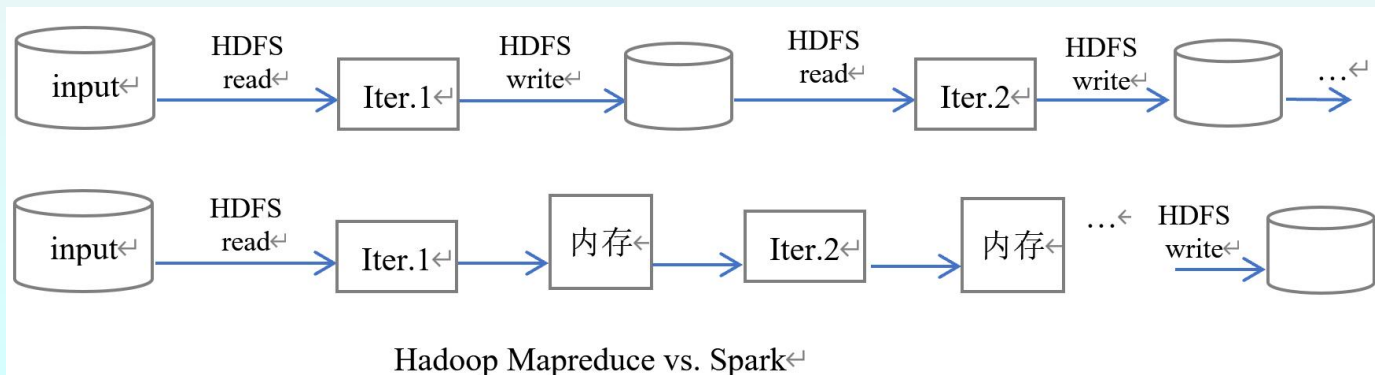
pause



Spark简介

◆ Spark特性总结

- ◆ (1) Spark的数据处理速度比Hadoop要快很多，这主要得益于其基于内存的数据处理技术
- ◆ Spark本身支持内存数据集的处理，以及存储在磁盘上的数据集的处理。Spark尽量把更多的数据驻留在内存中，必要的时候才需要存取磁盘
- ◆ 此外，对于中间结果，Spark也把它保存在内存中，而不是写入磁盘，对于需要对数据进行多次处理的计算任务来讲，避免I/O操作，可以极大提高处理效率
- ◆ Spark的基于内存的数据处理技术，使得它获得比其它大数据处理框架更高的性能



Spark简介

◆ Spark特性总结

- ◆ (2) Spark和Hadoop生态系统兼容，能够集成和操作HDFS、Amazon S3、Hive、HBase、Cassandra等数据源；Spark可以运行在Hadoop YARN或者Apache Mesos上，也可以独立运行(Standalone Mode)
- ◆ (3) 除了Map函数和Reduce函数，Spark提供其它大量的数据原语操作，使用这些操作可以表达复杂的计算任务；这些操作包括转换(Transformations)操作和动作操作(Actions)。
- ◆ (4) Spark可以对用DAG表达的复杂算法进行优化
- ◆ (5) 对于数据查询操作，Spark采用延迟计算的方式(Lazy Evaluation)来执行，从而帮助优化器对整个数据处理的工作流进行优化
- ◆ (6) Spark系统使用Scala语言进行编写，经过编译以后，在JVM上运行(Java Virtual Machine)；Spark系统提供了简洁、一致的编程接口，支持各种编程语言；目前Spark支持的开发语言包括Scala、Java、Python、Clojure、R等
- ◆ (7) Spark提供了一个交互式的Shell，使得用户可以使用Scala或者Python语言，交互式地对数据进行操作，马上看到结果

Spark简介

- ◆ **Hadoop的局限性和Spark的诞生**
 - ◆ Hadoop作为大数据处理的有效工具，已经存在了十年，并且从1.0版本，发展到了2.0版本以及3.0版
 - ◆ 传统的Hadoop平台使用MapReduce计算模型对数据进行处理
 - ◆ 对于只需要一遍扫描处理的计算任务(One Pass Computation)来讲，MapReduce计算模型非常有效
 - ◆ 但是某些计算任务，需要在数据上，进行多遍扫描和处理(Multi Pass Computation)，MapReduce 计算模型的效率就大大降低了
 - ◆ 在Hadoop/MapReduce平台上，数据处理流程，分为Map阶段和Reduce阶段，人们需要把任何计算任务转换成MapReduce计算模式，才能使用这套系统
 - ◆ 也就是当我们需要执行某些复杂的数据处理任务时，需要把它翻译成一系列的MapReduce作业，然后依次执行这些作业
 - ◆ 在计算过程的各个步骤之间，各个作业(Job)输出的中间结果，需要存储到分布式文件系统，然后才能被下一个步骤使用
 - ◆ 因为多副本容错复制和磁盘存取的关系，Hadoop执行翻译成多个MapReduce作业的计算任务的时候，效率不高

Spark简介

- ◆ **Hadoop的局限性和Spark的诞生**
 - ◆ **从计算模型来看**，Spark把计算任务表达成有向无环图作业，从而把整个数据处理流程的多个步骤完整地表达出来
 - ◆ 所以Spark能够支持对数据进行复杂的处理操作，无需像Hadoop那样，把复杂任务分解成一系列的MapReduce作业
 - ◆ Spark支持把数据驻留在内存中，作业(表达为DAG形式)的多个阶段间，实现数据不落地(写入磁盘)的流水线(Pipeline)处理
 - ◆ 所以，Spark不仅能够用DAG更加自然地表达各种复杂计算，**而且其性能也比Hadoop高**

Spark简介

- ◆ **Hadoop的局限性和Spark的诞生**
 - ◆ 并不是说，Hadoop大数据处理系统毫无意义
 - ◆ 从某种意义上来讲，正是Hadoop的成功和不足，催生了Spark这样的系统
 - ◆ 而Hadoop系统本身也在不断发展，包括新的计算模型的支持，比如流数据处理、图数据处理、以及DAG表达的计算任务(Tez系统)的处理
 - ◆ 所以，我们应该把Spark看作是Hadoop的一个备选项，而不是完全地替代Hadoop本身
 - ◆ 目前，Spark可以在Hadoop分布式文件系统HDFS上运行，把HDFS作为数据存储使用
 - ◆ Spark应用程序，可以部署到Hadoop2.0 YARN集群、或者Apache Mesos集群上，YARN和Mesos是可以互相替换的资源管理和调度软件(模块)

Spark简介

◆ Spark简介

- ◆ 相对于Hadoop系统，Spark主要优势包括

- ◆ **(1) 数据类型与计算的表达能力：**首先，Spark可以管理各种类型(Variety)的数据集，包括文本数据、图数据等；它是一个通用的平台，支持以有向无环图(Directed Acyclic Graph, DAG)形式表达的复杂的计算，支撑批处理、流数据处理、图数据处理、机器学习等众多应用场景

- ◆ **(2) 数据处理速度：**Spark基于内存计算技术进行数据处理，可以快速处理各种类型的数据集，降低处理延迟

- ◆ 当数据完全驻留于内存时，Spark的数据处理速度达到Hadoop系统的几十到上百倍

- ◆ 当数据保存在磁盘上时，需要从磁盘装载数据以后才能进行处理，它的处理速度能够达到Hadoop系统的10倍左右

- ◆ 注意，Spark不能完全替代Hadoop，它运行在Hadoop(HDFS)之上

pause



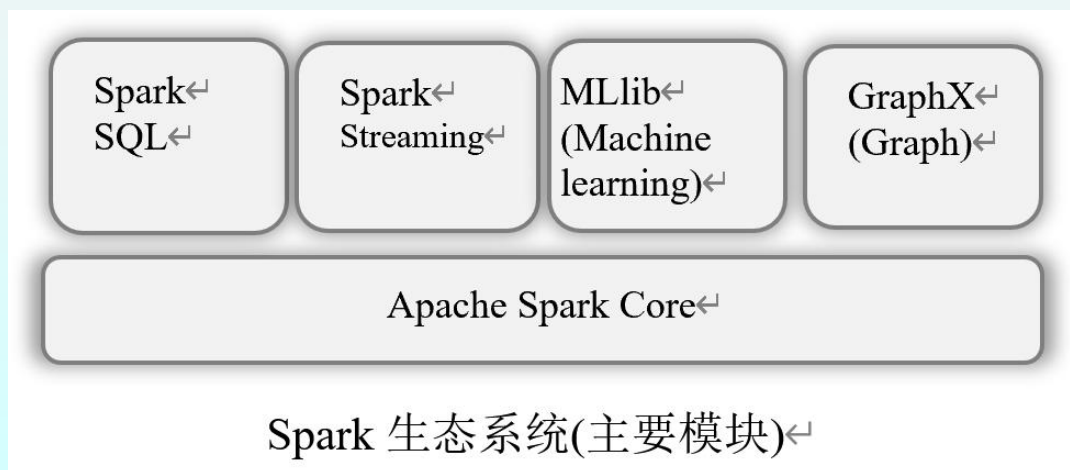
Spark简介

◆ Spark生态环境

- ◆ 整个Spark软件系统，包含核心模块(Spark Core)，以及若干数据处理分析模块
- ◆ 这些模块，一起构成整个Spark大数据处理的生态环境

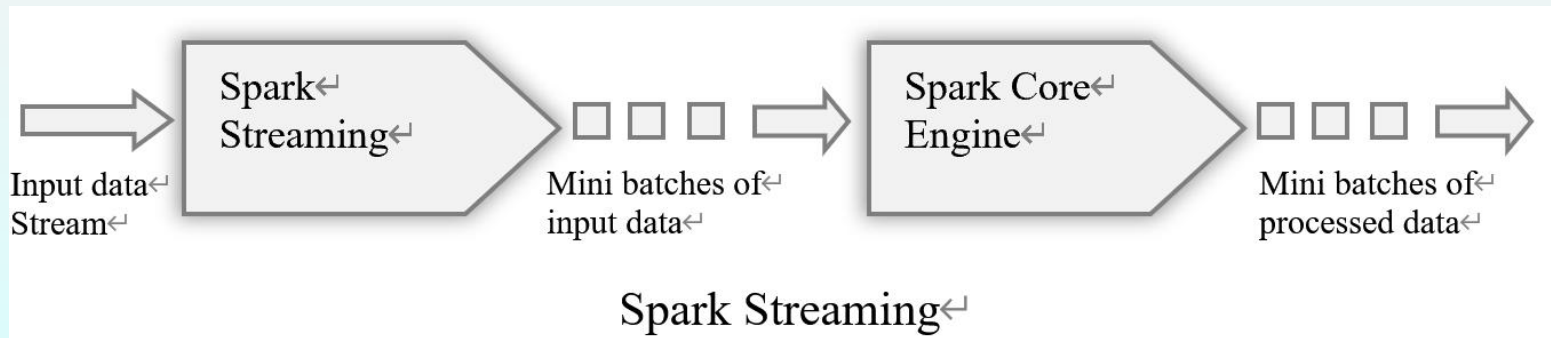
◆ Spark核心模块(Spark Core)

- ◆ Spark核心模块，是整个系统进行大规模并行和分布式数据处理的基础
- ◆ 它的主要功能是，内存管理和容错保证、集群环境下的作业调度和监控、以及和存储系统的接口和交互等。核心模块上的其它模块，介绍如下



Spark简介

- ◆ Spark生态环境
- ◆ 流数据处理模块(Spark Streaming)
 - ◆ 用于处理实时流数据，比如Web服务器的日志文件(Log File)、社交媒体如Twitter、以及各种消息队列如Kafka消息队列等
 - ◆ 它采用小批量(Micro Batch)数据处理的方式，即把接收的数据流，分解成一系列的小的RDD(下文介绍)，交给Spark引擎进行处理，实现流数据处理，其结果也是以批量的方式生成的(In Batch)



Spark简介

- ◆ Spark生态环境
- ◆ SQL查询与结构化数据处理(Spark SQL)
 - ◆ 该模块把Spark数据集通过JDBC API暴露出来，让客户程序可以在上面运行SQL查询；也可以把传统的BI(Business Intelligence)和可视化工具连接到该数据集，利用JDBC进行数据查询、汇总和展示等
 - ◆ Spark SQL模块支持不同外部数据源(比如JSON、Parquet列存储、以及关系数据库等)的导入、转换、和装载，并且支持即席(Ad-Hoc)查询
- ◆ 机器学习(Spark MLlib)
 - ◆ MLlib是Spark生态系统里可扩展的机器学习模块。它已经实现了众多的算法，包括分类(Classification)、聚类(Clustering)、回归(Regression)、协同过滤(Collaborative Filtering)、降维(Dimensionality Reduction)等
- ◆ 图数据处理(Spark GraphX)
 - ◆ GraphX支持图数据的并行处理。用户可以利用该模块，对图数据进行探索式分析(Exploratory Analysis)以及迭代式计算(Iterative Graph Computation)

Spark简介

- ◆ Spark生态环境
 - ◆ 此外，**BlinkDB**是一个近似查询处理引擎
 - ◆ 可以在大规模数据集上，交互式地执行SQL查询。这个查询引擎，允许用户在查询的精度和响应时间之间做出折中。用户可以指定查询响应时间、或者查询结果精度要求，BlinkDB在数据采样(Data Sample)上执行查询，获得一个近似结果。查询结果打上一个错误范围(Error Bar)标签，帮助用户做出决断
 - ◆ 整个Spark生态环境，还集成了一系列的**适配软件模块**，包括Spark Cassandra Connector、SparkR等，以支持Spark系统和Cassandra、R等软件系统的协同工作。比如通过Cassandra Connector，我们可以使用Spark去存取存在Cassandra数据库里的数据，进行必要的数据分析

pause



Spark简介

◆ **RDD弹性分布式数据集**

- ◆ 弹性分布数据集(Resilient Distributed Dataset, RDD)是Spark软件系统的核心概念
- ◆ 它是一个容错的、不可更新的(Immutable)分布式数据集，支持并行处理
 - ◆ 简单来讲，RDD可以看作数据库里的一张表
 - ◆ Spark把隶属于一个RDD的数据，划分成不同的分区(Partition)，分区是RDD的下级概念
 - ◆ 对RDD进行分区，分布到集群环境(不同节点)，有利于对数据进行并行处理
 - ◆ 我们可以从外部数据源比如HDFS、HBase等创建RDD数据集，也可以在Spark程序中，直接创建RDD，比如把一个List转换成一个RDD

Spark简介

- ◆ RDD弹性分布式数据集
 - ◆ 容错机制
 - ◆ RDD采用基于血统(Lineage)的容错机制
 - ◆ 也就是它记住每个RDD，是如何从其它RDD转换(Transformation)而来的
 - ◆ 当某个RDD损坏的时候，Spark系统从上游RDD重新计算和创建本RDD的数据

Spark简介

- ◆ RDD弹性分布式数据集
 - ◆ 操作Operation
 - ◆ RDD数据集是不可更新的(Immutable)
 - ◆ 我们可以对一个RDD进行转换(Transformation), 但是转换之后会返回一个新的RDD, 而原来的RDD保持不变。
 - ◆ RDD数据集支持两种操作, 分别是转换(Transformation)和动作(Action)
 - ◆ 对一个RDD施加转换操作, 将返回一个新的RDD(New RDD)
 - ◆ 典型的转换操作包括map、filter、flatMap、groupByKey、reduceByKey、aggregateByKey、pipe、以及coalesce等操作
 - ◆ 动作操作施加于RDD数据集, 经过对RDD数据集的计算, 返回一个新的结果(New Result), 这个结果将返回客户端
 - ◆ 典型的动作包括reduce、collect、count、first、take、countByKey、以及foreach等操作

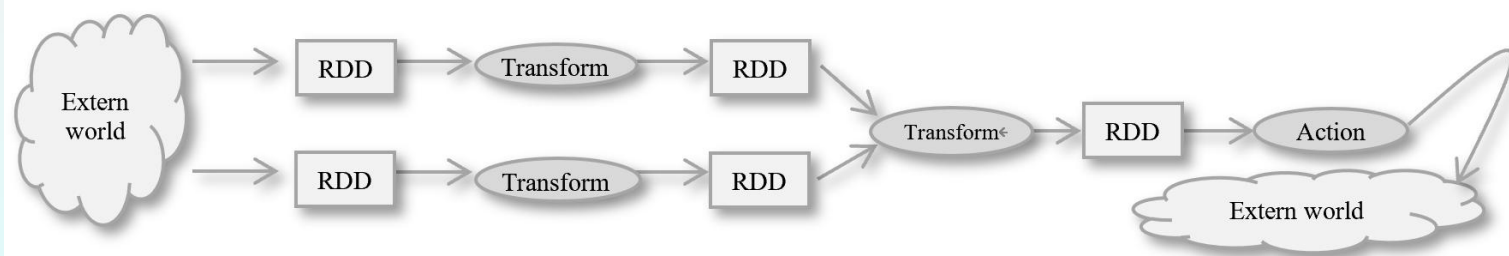
Spark简介

- ◆ RDD弹性分布式数据集
 - ◆ **Lazy Execution**
 - ◆ 转换操作是延迟(Lazy)执行的，也就是这个操作不会马上执行
 - ◆ 当某个动作(Action)操作被一个驱动程序(Driver Program，不是设备驱动程序，而是客户端软件)调用DAG的动作操作(Action)的时候，动作操作的一系列前导(Proceeding)转换操作，才会被启动
 - ◆ 这个特性使得Spark可以为DAG生成一个优化的执行计划

Spark简介

◆ DAG

- ◆ RDD、转换操作、动作操作等，一起构成一个 DAG，用以表达复杂的计算
- ◆ 一般来讲，对DAG中的每个RDD，当需要上面执行某个转换/动作的时候，将重新从上游RDD进行计算
- ◆ 我们也可以对RDD进行缓存、或者持久化
- ◆ 在这种情况下，Spark会保留这个RDD，以便后面可以再次存取它，从而获得更高的查询速度；缓存指的是把数据缓存在内存中

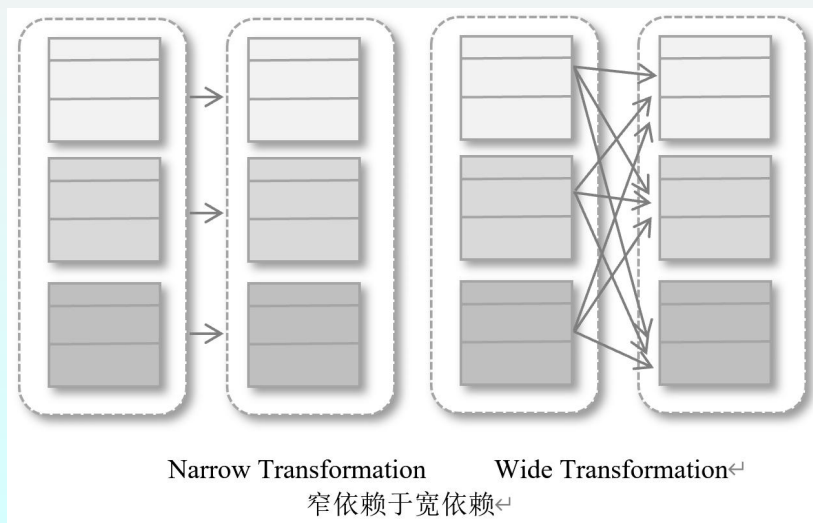


一个典型的 DAG

Spark简介

◆ 宽依赖有窄依赖

- ◆ 在DAG里，父子RDD的各个分区(Partition)之间，有两种依赖关系，分别是宽依赖和窄依赖
- ◆ 窄依赖，指的是每个父RDD的分区，最多被一个子RDD的分区使用到(父RDD的某个分区的数据经过转换操作，产生子RDD的分区)
- ◆ 而宽依赖，指的是多个子RDD的分区，依赖于同一个父RDD的分区的情形，groupByKey、reduceByKey、sortByKey等操作，需要用到宽依赖，以便获得正确的结果
- ◆ 请读者思考为什么？



Spark简介

- ◆ Filter过滤：窄依赖

- ◆ Partition0

key	value
holden	likes coffee
panda	likes long strings and coffee

窄依赖

filter

key	value
holden	likes coffee

- ◆ Partition1

key	value
holden	likes coffee
panda	likes long strings and coffee

filter

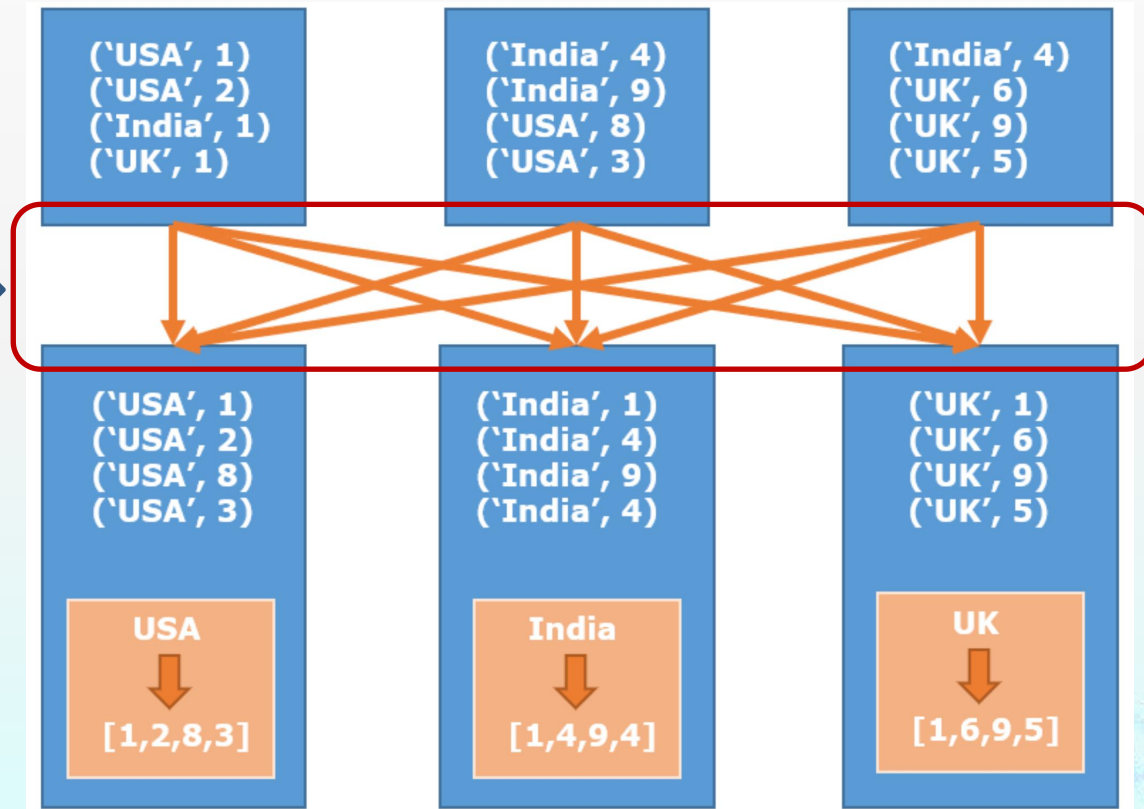
key	value
holden	likes coffee

Spark简介

- ◆ Group by 分组

- ◆ 宽依赖

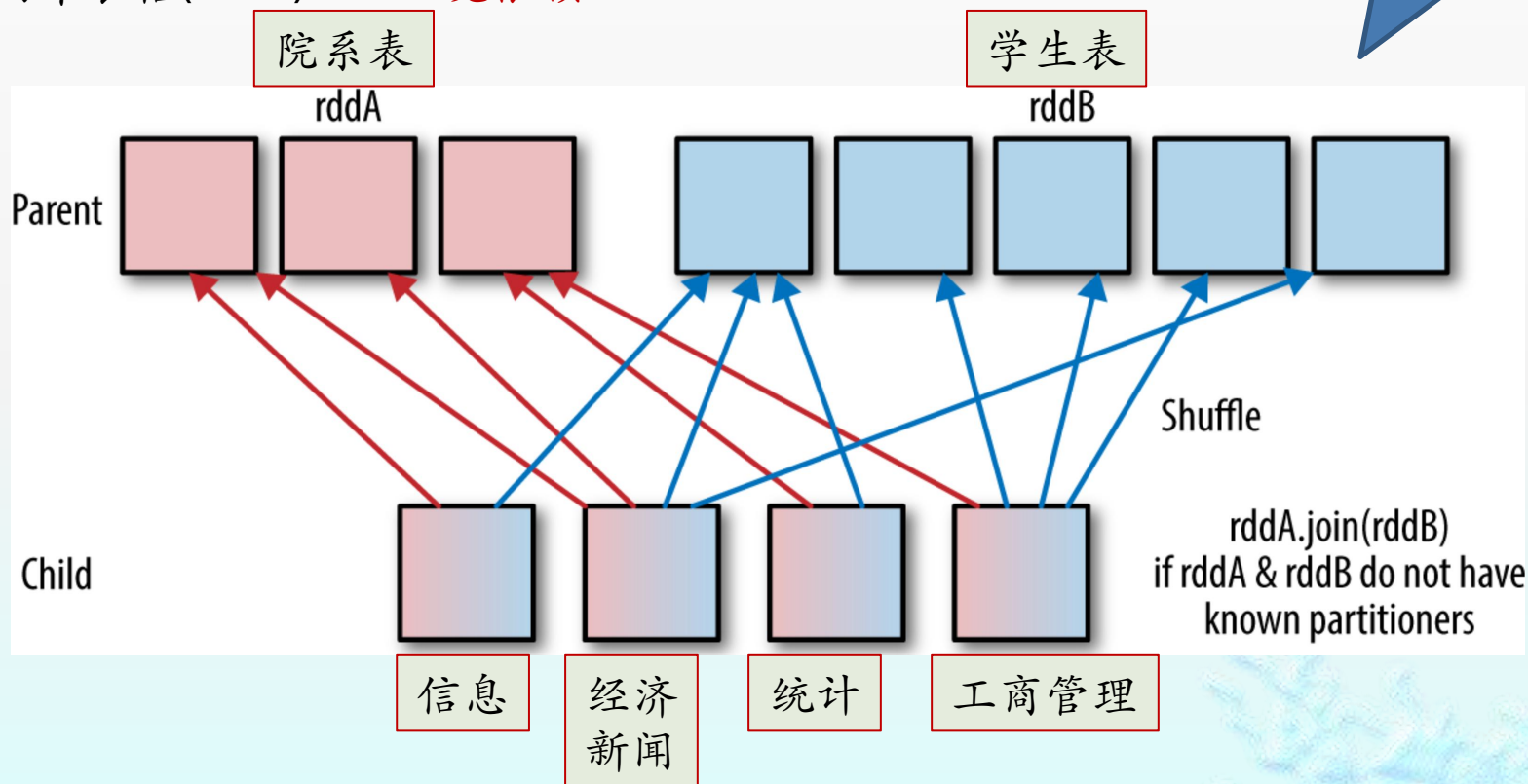
宽依赖



Spark简介

统计每个院系
的学生人数

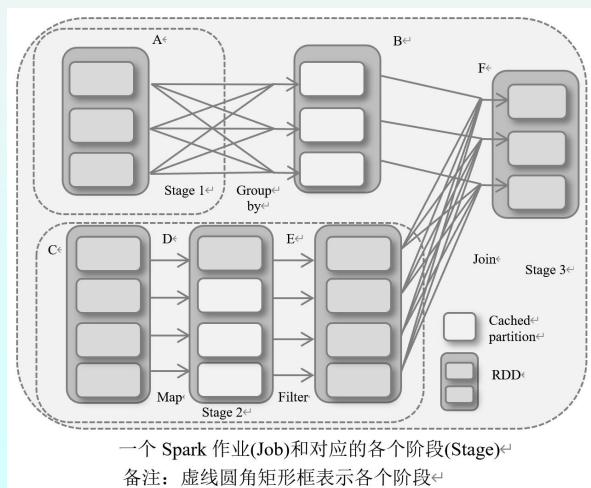
- 两个表格(RDD)Join: 宽依赖



Spark简介

◆ DAG的调度

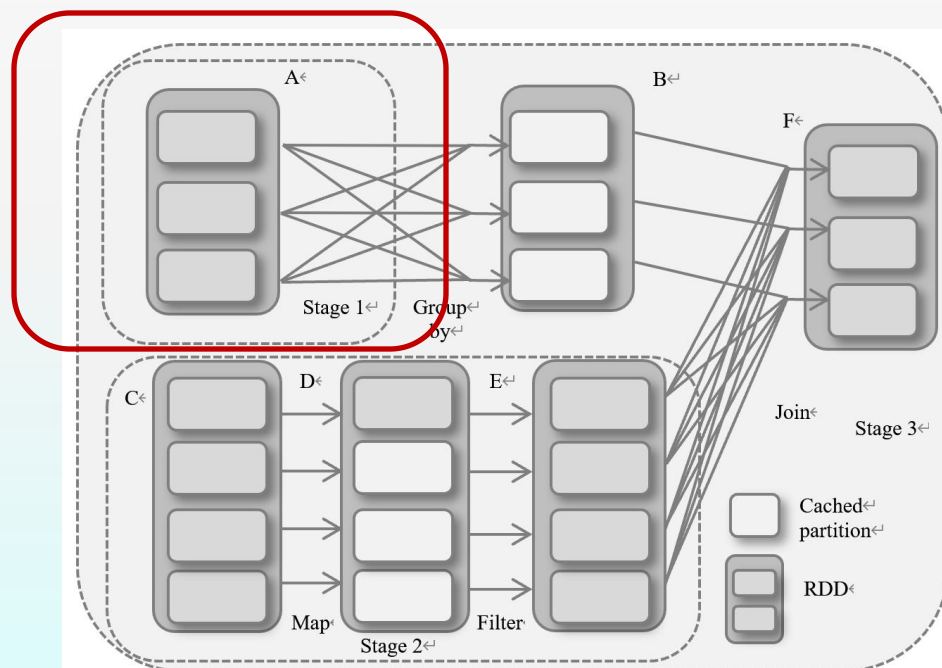
- ◆ DAGScheduler是面向阶段(Stage-Oriented)的DAG执行调度器
- ◆ DAGScheduler使用作业(Job)和阶段(Stage)等基本概念, 进行作业调度
- ◆ 一个作业, 是一个提交到DAGScheduler的工作项目(Work Item)。一个作业表达成一个DAG, 并且以一个RDD结束
- ◆ **阶段是一组并行任务**, 每个任务对应RDD的一个分区; 每个阶段是Spark作业的一部分, 负责计算部分结果, **它是调度的基本单元**
- ◆ DAGScheduler检查依赖的类型, 它把一系列窄依赖RDD组织成一个阶段; 而对于宽依赖, 则需要跨越连续的阶段



Spark简介

◆ DAG的调度

- ◆ 在Stage1里，对RDD A(是一张表，整个是一个RDD，划分成一系列分区，分布在各个节点上)进行Groupby处理，生成RDD B。RDD A和RDD B之间的依赖关系是宽依赖



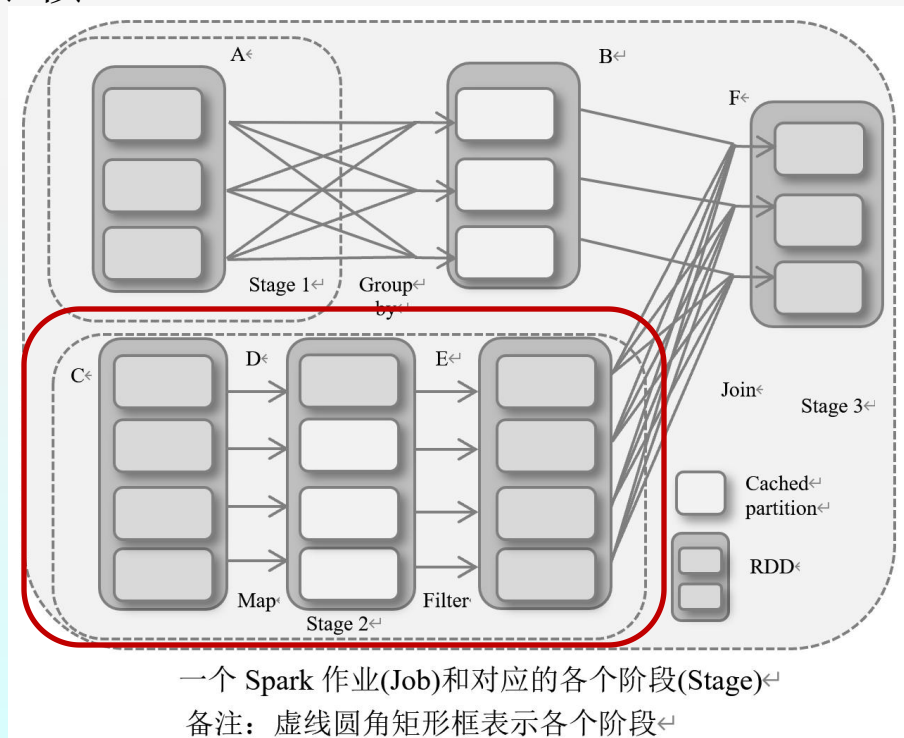
一个 Spark 作业(Job)和对应的各个阶段(Stage)

备注：虚线圆角矩形框表示各个阶段

Spark简介

◆ DAG的调度

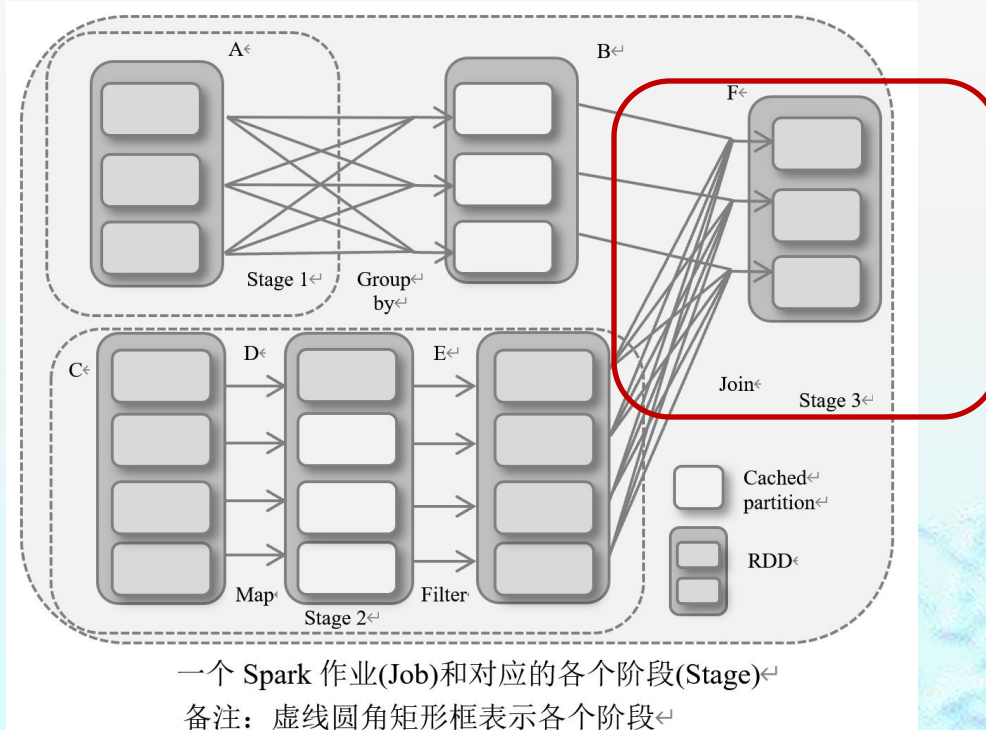
- ◆ 在Stage2里，对RDD C(是另外一张表，整个是一个RDD，划分成一系列分区，分布在各个节点上)进行map、filter等操作，依次生成RDD D、RDD E等中间结果，RDD C与RDD D之间、以及RDD D与RDD E之间的依赖关系是窄依赖



Spark简介

◆ DAG的调度

- ◆ 在Stage3里，对RDD B和RDD E做join操作。RDD B和RDD F之间的依赖关系是窄依赖，而RDD E和RDD F之间的依赖关系是宽依赖



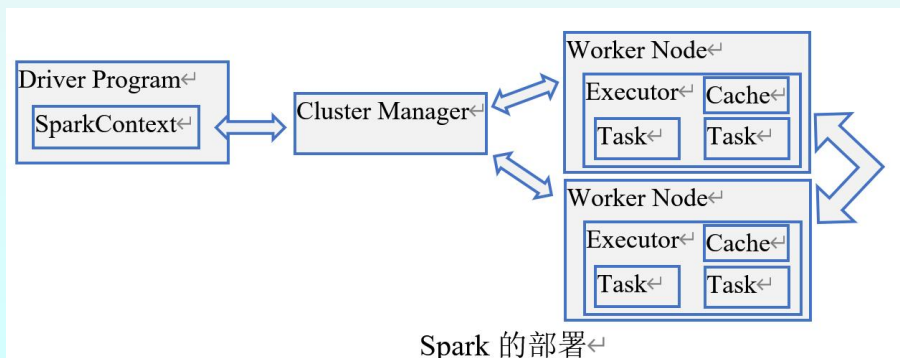
pause



Spark简介

◆ Spark部署

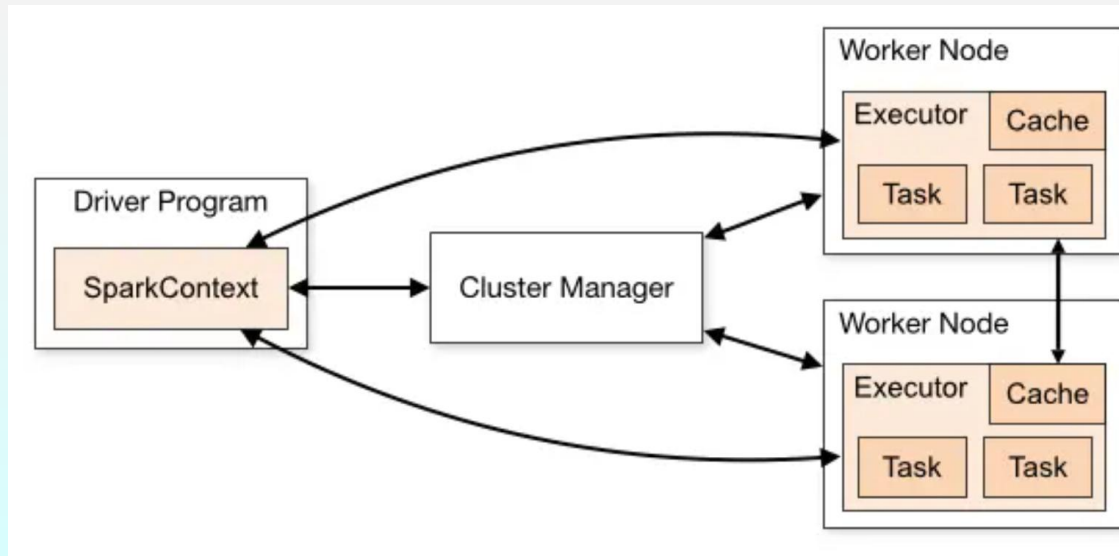
- ◆ **Driver**负责运行应用程序的main函数，创建SparkContext
- ◆ **Spark Context**持有到Cluster Manager的连接。每个Spark应用程序都是一组独立的进程，由Spark Context协调运行。当Spark Context连接到Cluster Manager以后，它申请获得从节点上的一系列Executor。这些Executor独立工作(运行Task)又相互沟通，完成整个作业(Job)，即应用程序的处理要求
- ◆ **Worker**是Cluster的从节点上运行的进程，用于执行运行应用程序。一个应用程序启动以后，它在各个Worker上申请Executors，来真正执行应用程序的处理逻辑(Task)
- ◆ **Cluster Manager**负责为应用程序分配资源。**Spark支持3种Cluster Manager，即Standalone、Mesos、和YARN**。这些Cluster Manager在调度算法、安全性、监控能力等方面有差别



Spark简介

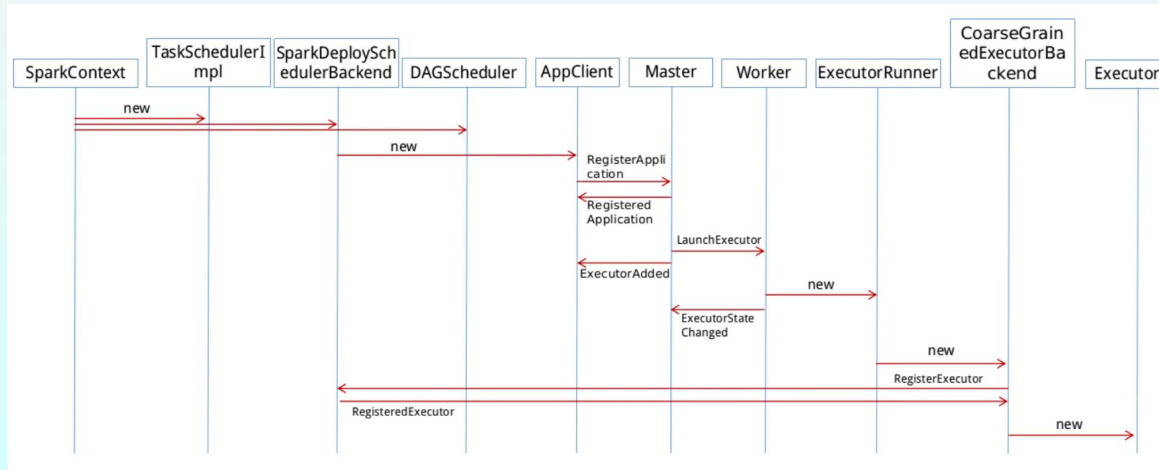
◆ Spark Job执行流程

- ◆ (1) 用户提交Job后会生成Spark Context对象
- ◆ (2) Spark Context向Cluster Manager（在Standalone模式下是Spark Master）申请Executor资源
- ◆ (3) Spark Context并将Job分解成一系列可并行处理的task，然后将task分发到不同的Executor上运行
- ◆ (4) Executor在task执行完后将结果返回到Spark Context



Spark简介

- ◆ **SparkContext**申请**Executor**资源的过程
- ◆ (1) **SparkContext**创建**TaskSchedulerImpl**, **SparkDeploySchedulerBackend**和**DAGScheduler**
- ◆ (2) **SparkDeploySchedulerBackend**创建**AppClient**, 并通过一些回调函数来得到**Executor**信息
- ◆ (3) **AppClient**向**Master**注册Application
- ◆ (4) **Master**向**Worker**发送**LaunchExecutor**消息, 同时向**AppClient**发送**ExecutorAdded**消息
- ◆ (5) **Worker**创建**ExecutorRunner**, 并向**Master**发送**ExecutorStateChanged**的消息
- ◆ (6) **ExecutorRunner**创建**CoarseGrainedSchedulerBackend**
- ◆ (7) **CoarseGrainedSchedulerBackend**向**SparkDeploySchedulerBackend**发送**RegisterExecutor**消息
- ◆ (8) **CoarseGrainedSchedulerBackend**在接收到**SparkDeploySchedulerBackend**发送的**RegisteredExecutor**消息后, 创建**Executor**



pause



Spark简介

◆ Spark的应用

- ◆ Spark生态系统的各个工具，可以有效地应对大数据的数据量(Volume)、速度(Velocity)、数据多样性(Diversity)等多重挑战，Spark胜任日志数据、电信基站数据.....等数据的处理和分析工作
- ◆ 下面列举部分可以使用Spark技术的实际应用
- ◆ **在游戏行业**，如果能够对游戏中的大量实时事件(Real Time Event)进行分析，并且发现一些模式，就可以进行快速响应，比如进行精准广告投放(Targeted Advertising)、玩家的挽留(Retention)、以及自动调整游戏的复杂度等。这些分析，将是非常有利可图的

Spark简介

◆ Spark的应用

- ◆ 在电商领域，实时的交易信息可以作为一个数据流，在数据流上可以运行一些聚类算法比如K-Means算法、或者协同过滤算法(Collaborative Filtering)，所获得的分析结果，可以结合其它的非结构化数据源，比如客户对商品的评价信息等，用于不断优化和调整展示给用户的推荐信息
- ◆ 在金融以及网络安全领域，Spark生态系统工具可以用于欺诈检测(Fraud Detection)、入侵检测(Intrusion Detection)等。通过对大量的日志进行分析，并且和外部数据源进行合并，比如关于数据泄漏、受损账户、连接和请求发出的IP地址所在位置、以及时间等信息，我们就可以获得更加准确的预测效果

Spark简介

<http://bigdata.stratebi.com/>

◆ Spark的应用



Real Time

See Demos



Social Network

See Demos



Big Data Analytics

See Demos



Internet of Things

See Demos



Graph Analysis

See Demos



Machine Learning

See Demos



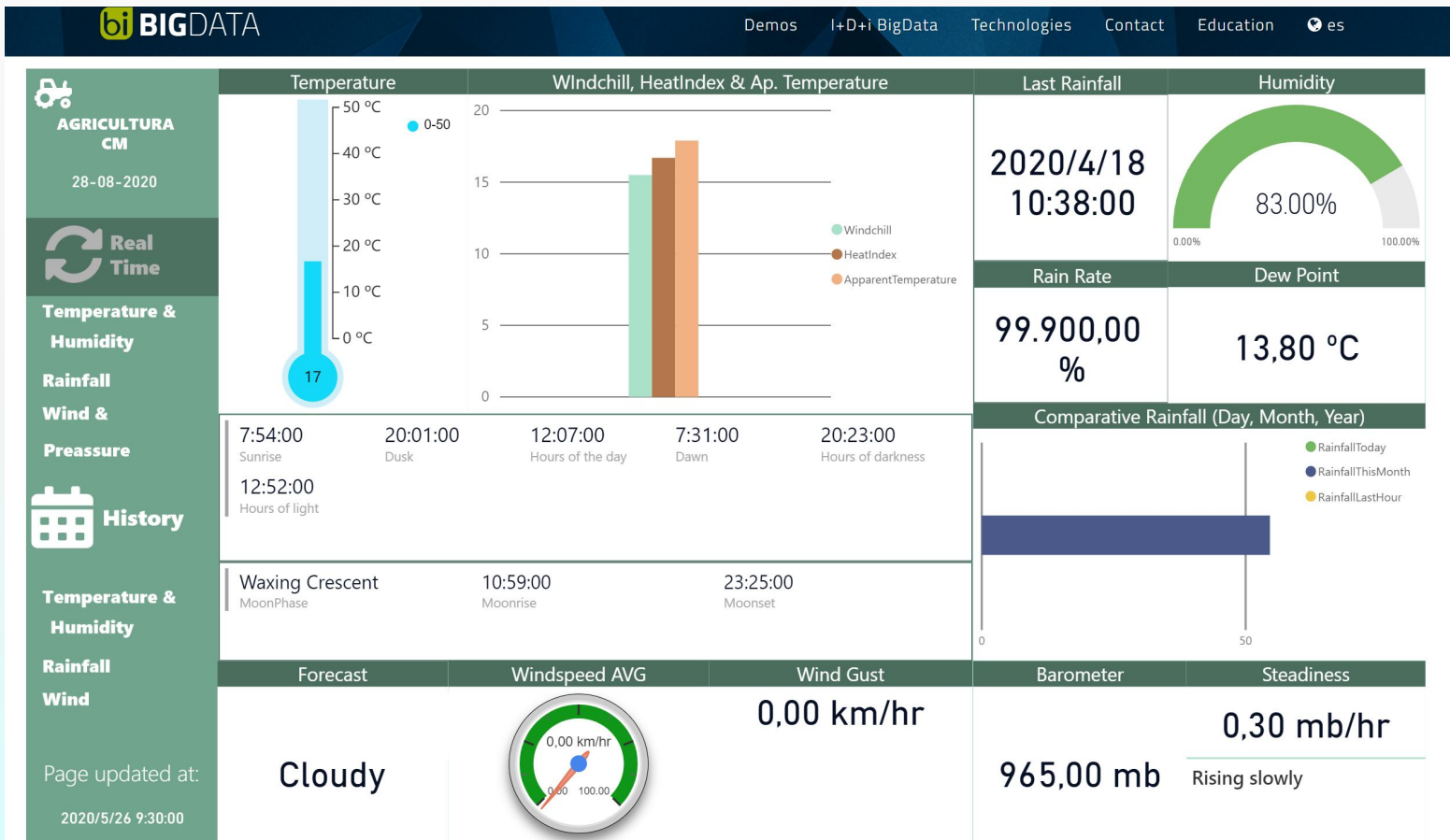
Web Semantic

See Demos

Spark简介

<http://bigdata.stratebi.com/>

◆ Spark的应用

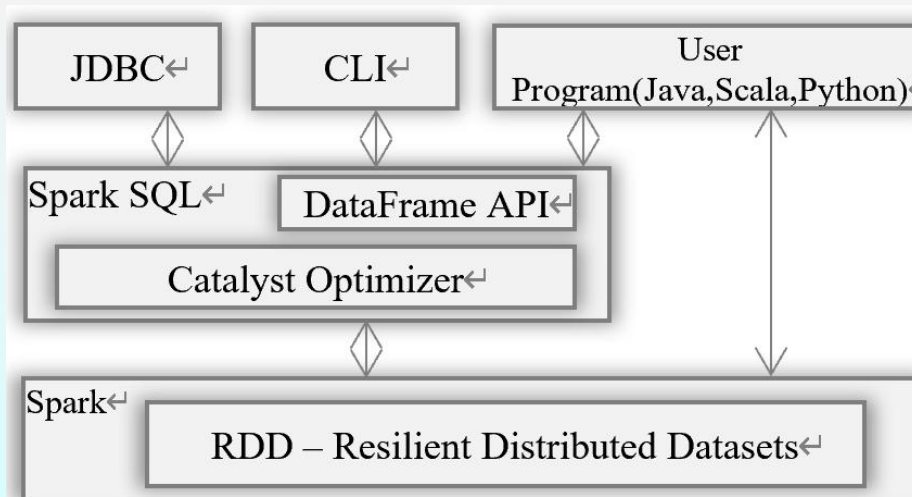


pause



Spark简介

- ◆ **SparkSQL**
- ◆ SparkSQL是Spark大数据处理框架的重要组成部分；用户可以对结构化数据用SQL语言进行查询分析
- ◆ 用户可以通过ETL(Extraction Transformation, Loading)工具，从不同格式的数据源装载数据，然后运行一些即席查询(Ad-Hoc Query)
- ◆ SparkSQL和其它系统组件的关系如图所示，其中Catalyst为SparkSQL的查询优化器



SparkSQL 和其它系统组件的关系

Spark简介

- ◆ **SparkSQL的主要特性**

- ◆ **(1) DataFrame:** 新版本SparkSQL, 在数据存储层面通过数据模式(Schema)完成对数据的结构化描述, 称为DataFrame; DataFrame是SparkSQL的一个重要的软件抽象层
- ◆ DataFrame是给不同属性列命名(Named Column) (加上数据类型的描述)的分布式数据集, 它和关系数据库的数据库表结构非常类似
- ◆ DataFrame从SparkSQL前一个版本的SchemaRDD发展而来, 并且进行了大量的代码改写
 - ◆ SchemaRDD是在RDD的基础上, 增加了一个模式(Schema)层, 以便对数据集的各个数据列进行命名和数据类型描述

Spark简介

- ◆ **SparkSQL的主要特性**

- ◆ **(2) Data Source:** Spark增加了Data Source应用程序编程接口(Application Programming Interface, API)。通过这个API, SparkSQL可以存取以不同格式保存的结构化数据, 包括Parquet格式、JSON格式、以及Apache Avro 数据序列化格式(Data Serialization)等

- ◆ JDBC Data Source, 是SparkSQL支持的其中一个数据源类型。用户可以通过JDBC编程接口(API), 从关系数据库读取数据, 并且把读取的数据和其它的数据集进行连接操作(Join)

- ◆ **(3) JDBC服务器(Server):** Spark SQL内置一个JDBC服务器, 客户端程序可以通过JDBC驱动程序, 连接到该服务器, 存取SparkSQL数据库表